



UNIVERSIDADE FEDERAL DA PARAÍBA – CIÊNCIA DE DADOS PARA NEGÓCIOS
PROJETO ORIENTADO À OBJETOS

MR

Muscle Rank

SAÚDE E DIVERSÃO

***UM APP DE
GESTÃO DE
TREINOS
GAMEFICADO***



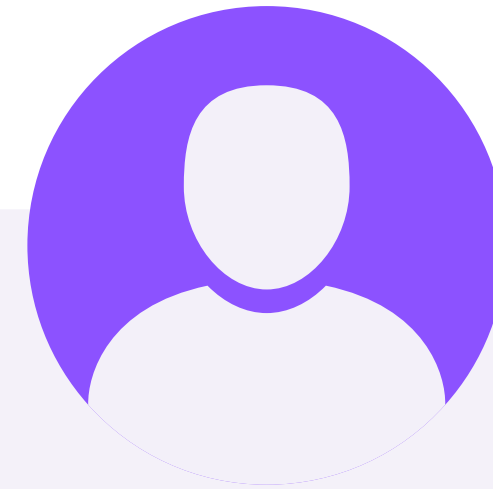
GRUPO:



ALEX
TAVARES



ARTHUR
SILVA



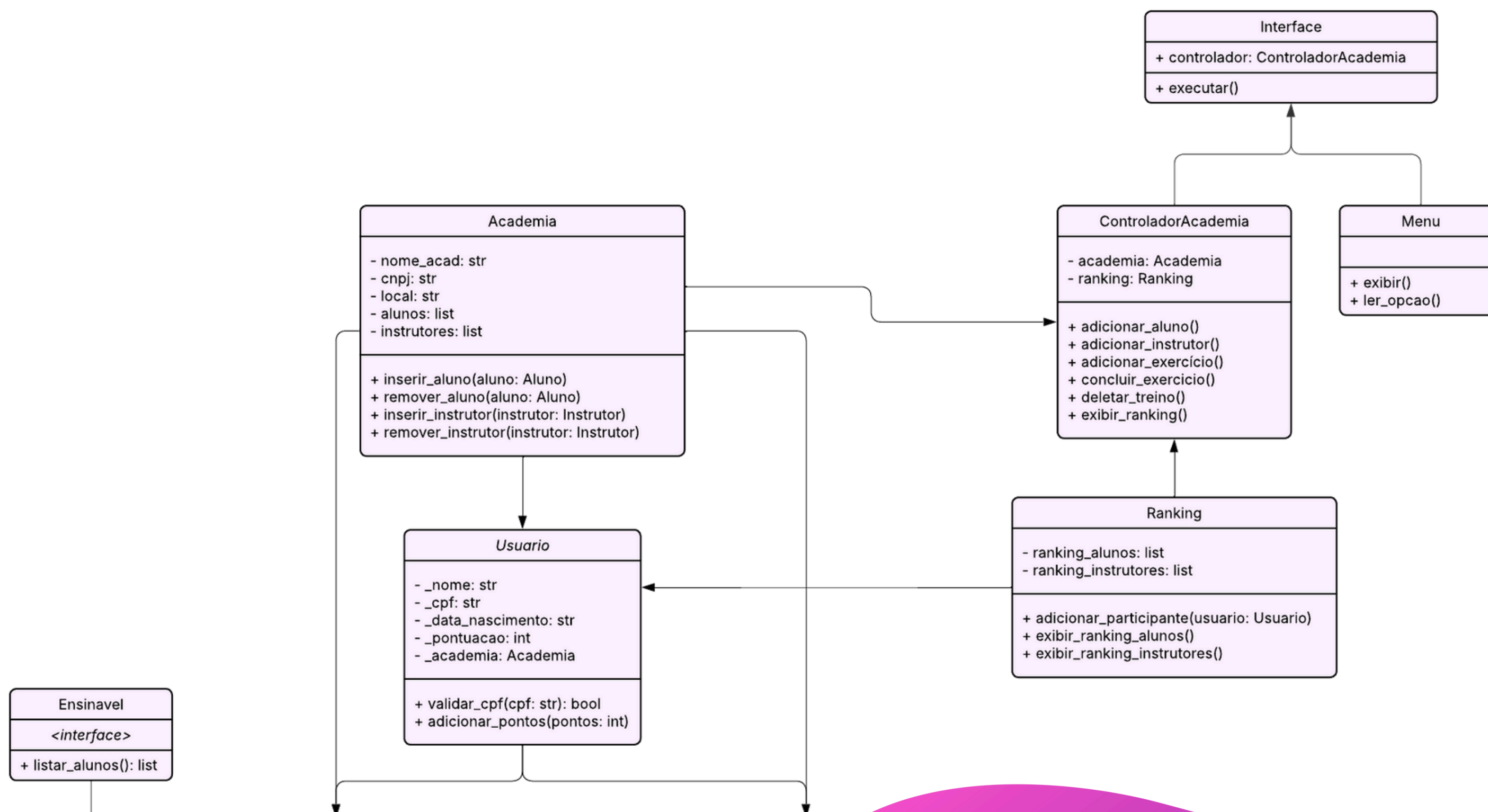
KAIO
VICTOR

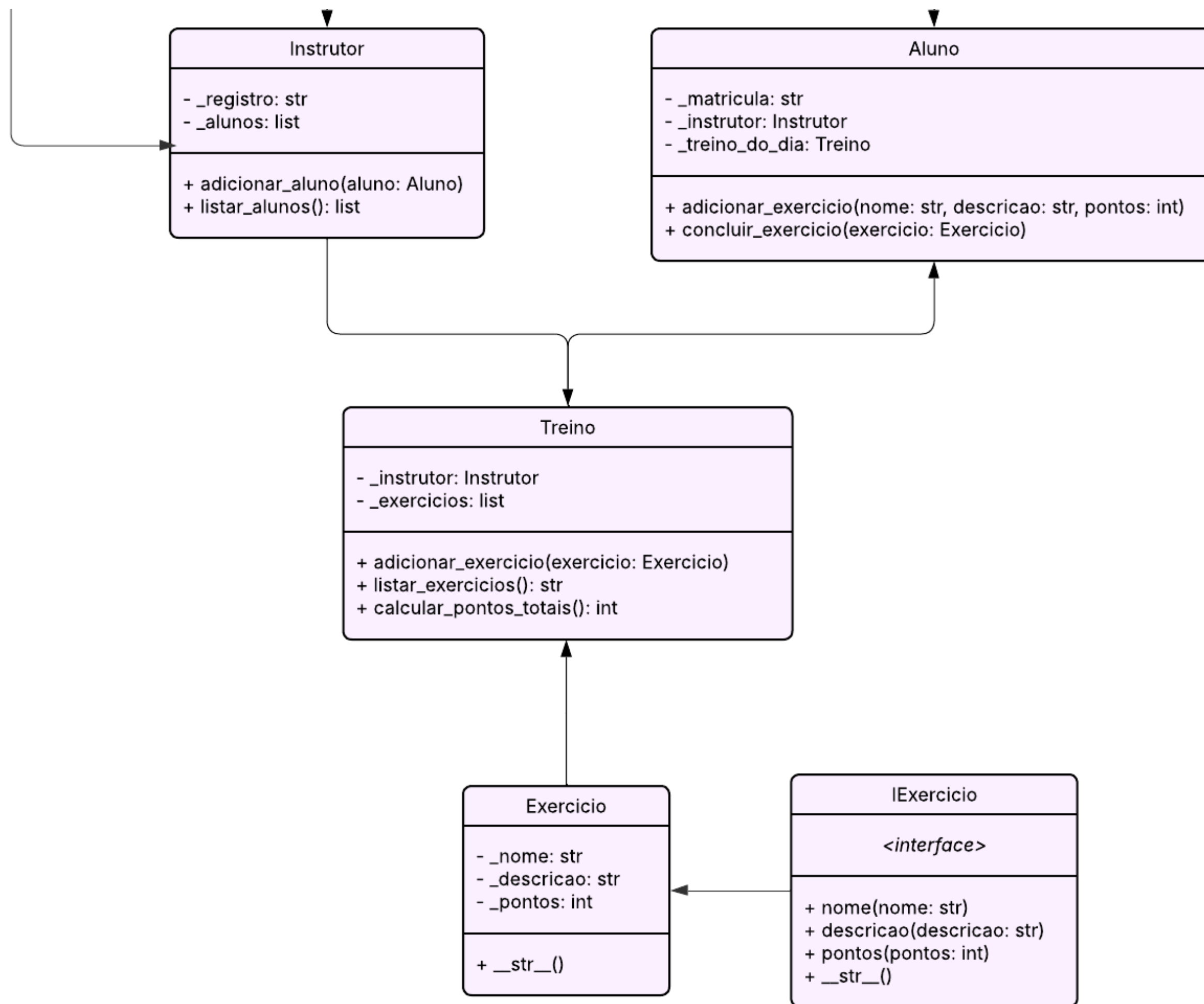
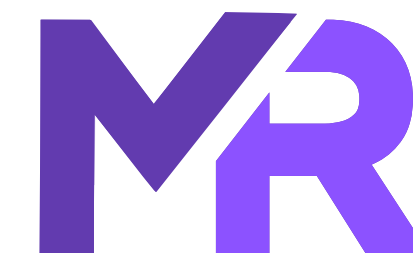
PROFESSORA: **ADRIANA**
DAMASCENO



Já pensou em deixar seu treino mais organizado e ao mesmo tempo mais divertido?

Nós somos a **Muscle Rank**, uma ferramenta dedicada a ajudar academias a terem alunos mais ativos e praticantes, tornando a atividade física mais divertida e competitiva. Através da gamificação, os alunos acumulam pontos ao realizar exercícios e treinos, competindo entre si com base em seus interesses e objetivos, enquanto disputam por recompensas motivadoras. Dessa maneira, estimulamos o engajamento e a constância nos treinos, transformando o hábito de se exercitar em uma experiência envolvente e desafiadora.





Serviços Oferecidos:

1. Adicionar Aluno
2. Adicionar Instrutor
3. Adicionar Exercício ao Treino do Dia
4. Concluir Exercício
5. Deletar treino
6. Exibir Ranking de Alunos
7. Exibir Ranking de Instrutores
8. Exibir Informações da Academia
9. Exibir Alunos da Academia
10. Exibir Instrutores da Academia
11. SAIR



Atualizações:

- Herança
- Classes Abstratas (NOVO)
- Composição
- Sobrecarga de Operadores
- Decoradores
- Encapsulamento
- Interfaces (NOVO)
- Tratamento de Exceções
- SOLID (NOVO)



Recaptulando:

Herança:

- Aluno e Instrutor herdam de Usuário.

Composição:

- Usuário possui um atributo que é uma instância da classe Academia.
- Treino contém Exercício em uma lista de exercícios.
- Aluno tem um Instrutor associado.

Sobrecarga de operadores:

- Utiliza o operador `_len_` para impedir que o usuário adicione mais que 10 exercícios ao seu treino do dia.



Recaptulando:

Encapsulamento e decoradores:

- Atributos de todas as classes principais, utilizando decoradores.

Tratamento de Exceções:

- Validar o CPF do usuário, instrutor não registrado, verificar se as instâncias de alguns objetos estão corretas.

Classes Abstratas:

A classe Usuário agora é uma classe abstrata, visto que não há instância direta dessa classe, apenas de suas subclasses Aluno e Instrutor. Assim, todos os métodos abstratos implementados serão obrigatórios nas subclasses, fazendo com que todos os usuários criados sigam um padrão mínimo.

```
1 from abc import ABC, abstractmethod
2 class Usuario(ABC):
3     def __init__(self, nome, cpf, data_nascimento, academia):
4         if not self.validar_cpf(cpf):
5             raise ValueError("CPF inválido!")
6         self._nome = nome
7         self._cpf = cpf
8         self._data_nascimento = data_nascimento
9         self._pontuacao = 0
10        from academia import Academia
11        if isinstance(academia, Academia):
12            self._academia = academia
13        else:
14            raise ValueError("O parâmetro 'academia' deve ser uma instância da classe Academia.")
```


Interfaces:

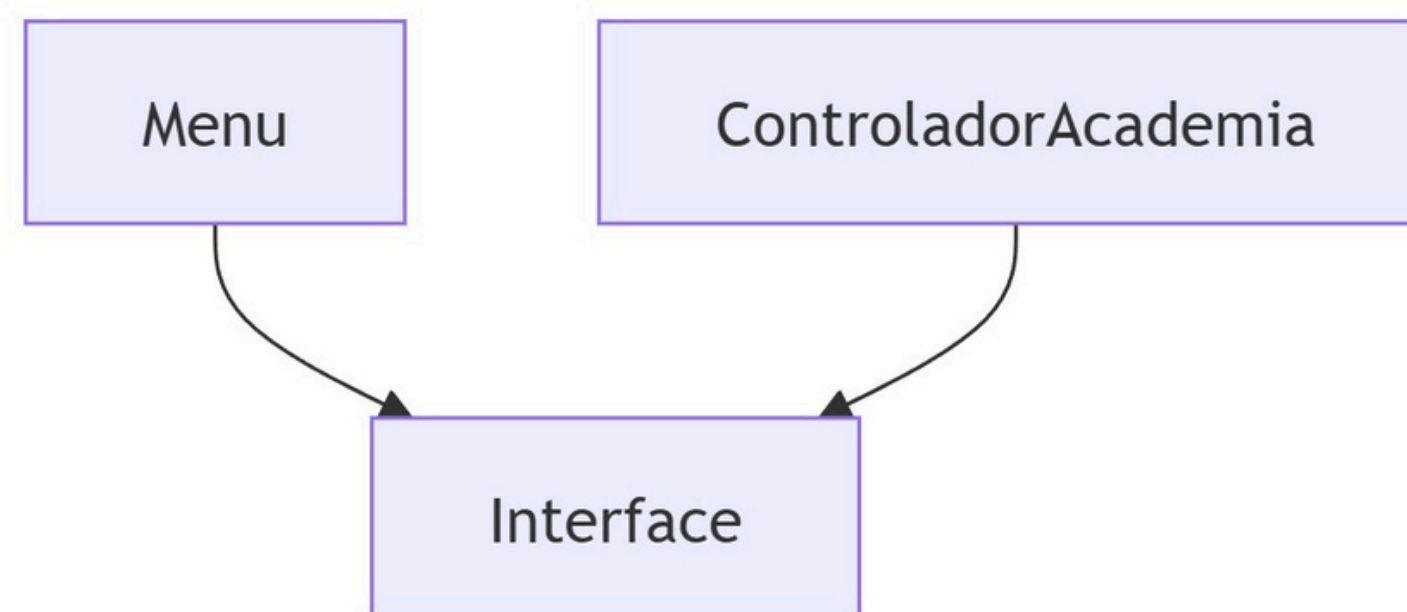
- Interface Ensinavel para Instrutor. Assim, apenas os instrutores podem executar o método listar_alunos.
- Interface IExercicio certifica que todos os exercícios sigam o mesmo padrão de informações, e a classe Treino verifica se a Interface foi implementada corretamente.

```
1 class Ensinavel(ABC):
2     @abstractmethod
3     def listar_alunos(self):
4         pass
5
6 class Instrutor(Usuario, Ensinavel):
```

```
1 from abc import ABC, abstractmethod
2
3 # Interface para exercícios
4 class IExercicio(ABC):
5     # ...
6
7 class Exercicio(IExercicio):
8     def __init__(self, nome, descricao, pontos):
9         self._nome = nome
10        self._descricao = descricao
11        self._pontos = pontos
12
13 class Treino:
14     def adicionar_exercicio(self, exercicio):
15         if isinstance(exercicio, IExercicio): # Verifica pela interface
16             self._exercicios.append(exercicio)
17         else:
18             raise ValueError("O objeto deve implementar a interface IExercicio.")
```

SOLID:

A classe Interface não seguia os princípios da SOLID (Single Responsibility Principle e Dependency Inversion Principle), então criou-se as classes Menu, ControladorAcademia e Interface, onde o Menu é uma classe responsável pelo menu inicial do sistema, o ControladorAcademia é responsável pela lógica de negócio (cadastro, treino e ranking), e a Interface é responsável pela interação com o usuário (entrada e saída).



Conclusão dos Métodos Aplicados:

A aplicação dos conceitos de Programação Orientada a Objetos em Python trouxe organização e padronização ao sistema. A classe Usuário foi tornada abstrata, impedindo sua instância direta e exigindo que Aluno e Instrutor implementem os métodos definidos, garantindo um comportamento mínimo comum.

A interface Ensinavel foi criada para restringir o método listar_alunos apenas aos instrutores, reforçando a separação de papéis dentro do sistema.

Os princípios do SOLID também foram aplicados: o Princípio da Responsabilidade Única foi atendido com a separação das classes Menu, ControladorAcademia e Interface, cada uma focada em uma responsabilidade específica. Já o Princípio da Inversão de Dependência permitiu um código mais flexível e desacoplado, facilitando testes e futuras alterações.

Com isso, o sistema se tornou mais modular, reutilizável e fácil de manter, respeitando boas práticas de desenvolvimento orientado a objetos.

Dúvidas e Dificuldades:

Durante o projeto encontramos...





Obrigado pela
atenção!

[Clique aqui para obter o código!](#)